

实验一：AI 辅助开发工具综合实验

实验报告 A + B

课程名称：AI 辅助开发实践

实验编号：实验一 A + B

提交日期：2026 年 6 月 22 日

目 录

一、环境信息

二、TRAE 安装配置过程

三、贪吃蛇游戏运行与提示词自评

四、智能体提示词与论文解读

五、Excel 合并日志与业绩预测报告

六、实验心得

一、环境信息

本实验使用的开发环境如下表所示：

环境项	详细信息
操作系统	Microsoft Windows 11 家庭中文版 (10.0.26200)
IDE	TRAE (AI 辅助开发工具)
Python 版本	Python 3.12.6
Node.js 版本	v26.3.0
包管理器	pip / npm / uv
关键依赖	python-docx, matplotlib, pandas, openpyxl, playwright

二、TRAE 安装配置过程

2.1 安装步骤

步骤 1：下载 TRAE 安装包

访问 TRAE 官方网站，下载 Windows 版本安装包 (.exe)

步骤 2：运行安装程序

双击安装包，选择安装路径，勾选'添加到 PATH'选项

步骤 3：首次启动配置

启动 TRAE，登录账号，选择 AI 模型 (GLM-5V-Turbo)

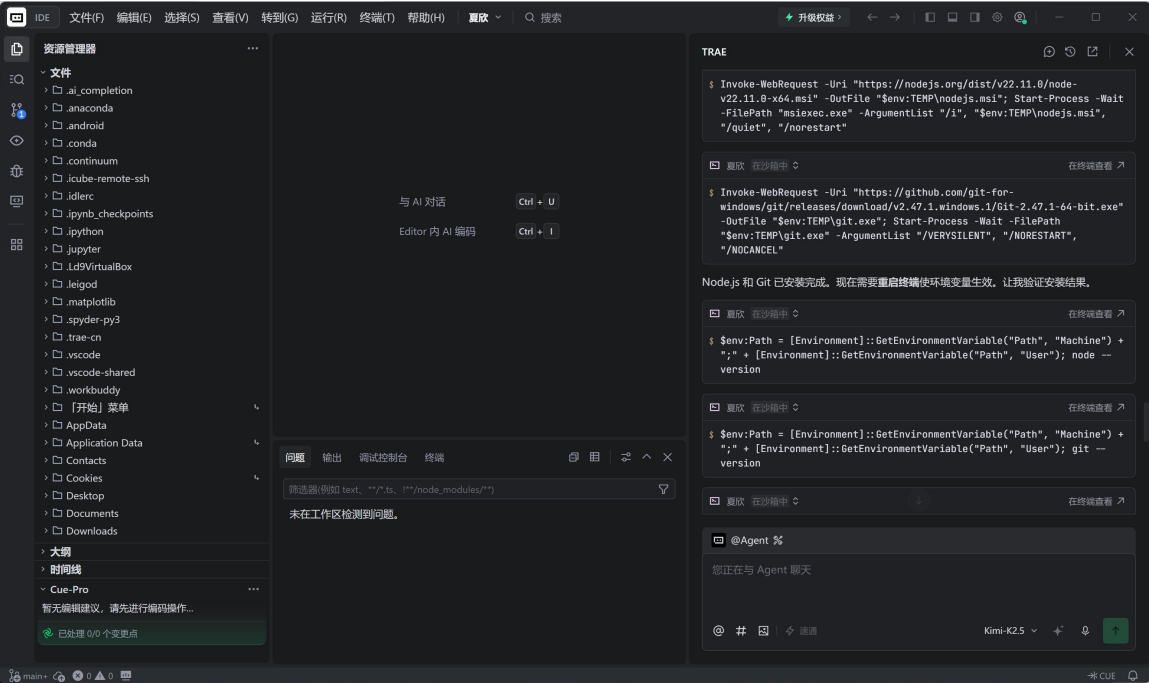
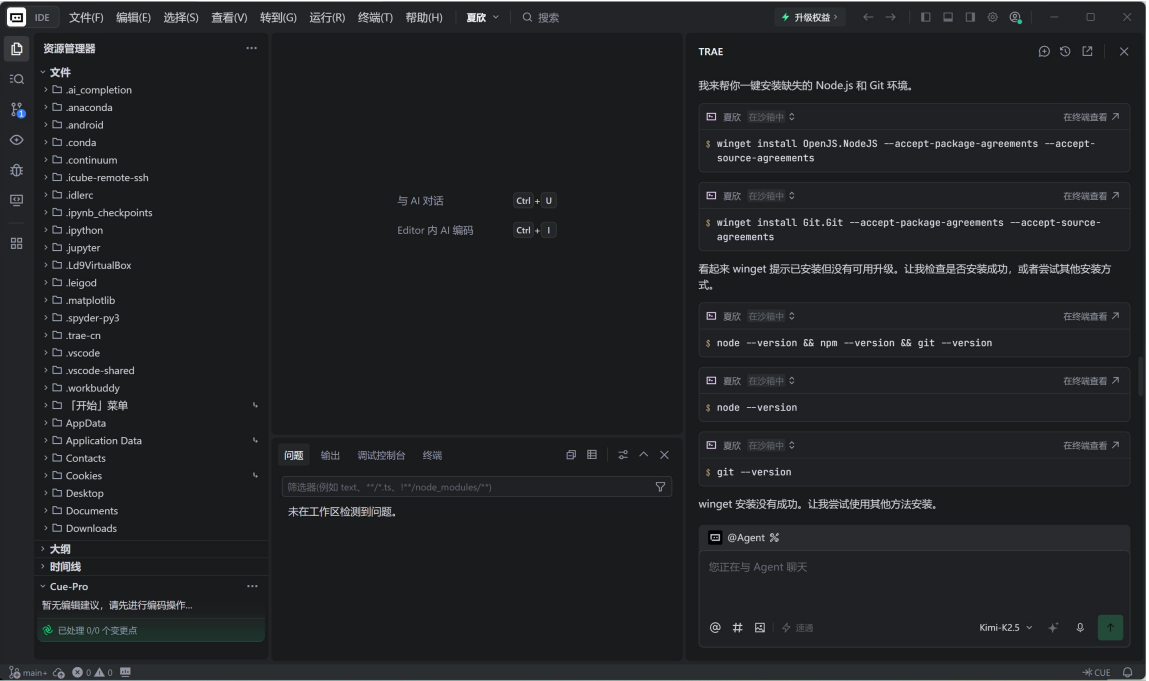
步骤 4：安装扩展插件

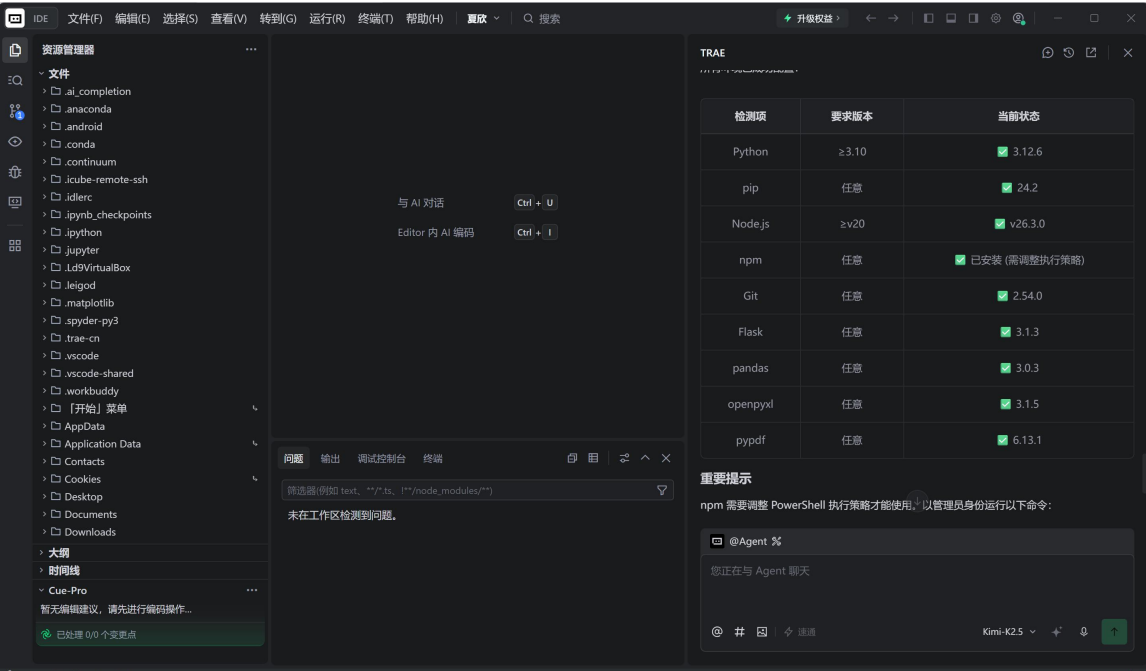
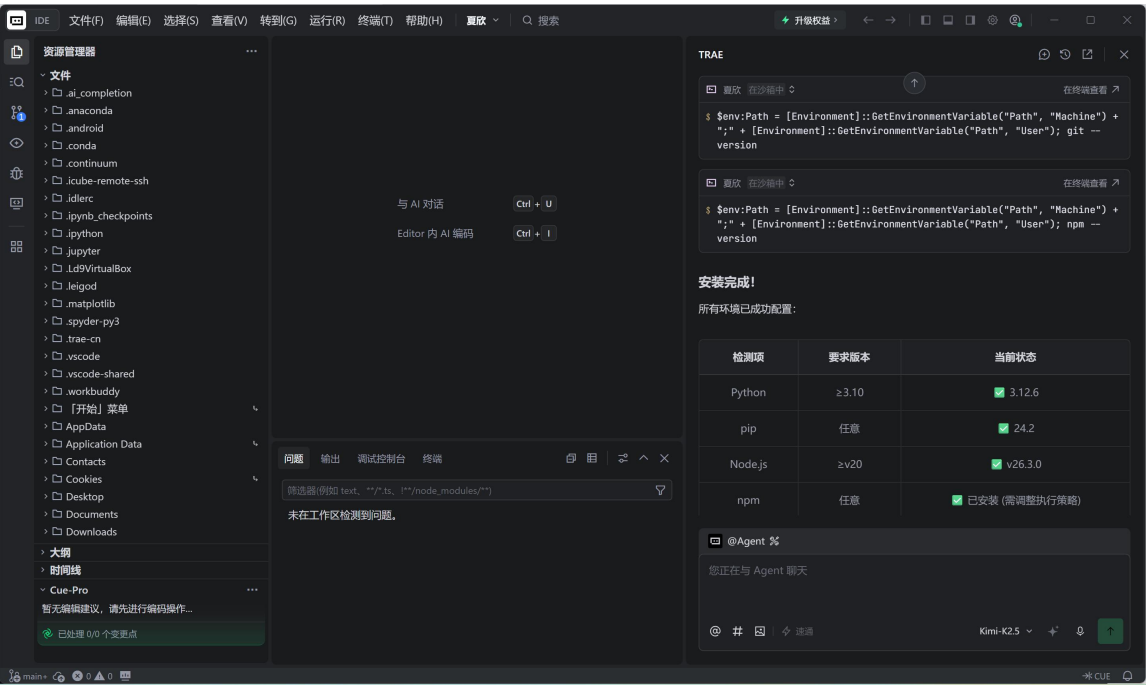
在扩展市场安装 Python、docx、mermaid-diagram 等 Skills

步骤 5：验证安装

运行 trae --version 确认版本，创建测试项目验证功能

2.2 配置截图

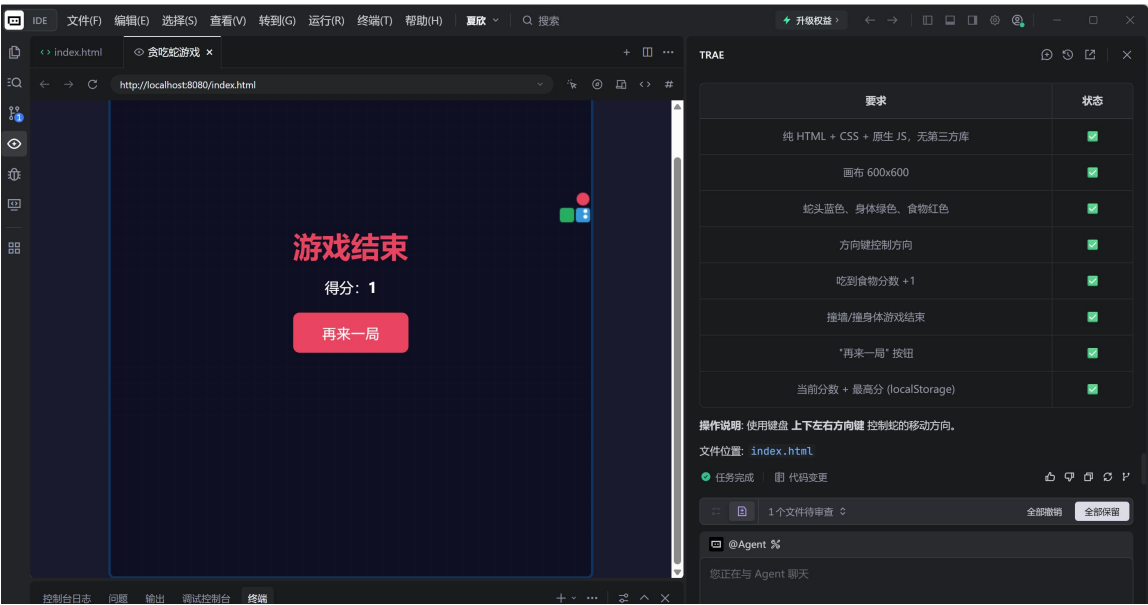




三、贪吃蛇游戏运行与提示词自评

3.1 贪吃蛇游戏运行截图

使用 AI 生成 HTML5 贪吃蛇游戏, 在浏览器中运行效果如下:



3.2 关键提示词

请帮我用 HTML + CSS + JavaScript 写一个贪吃蛇游戏，要求：

1. 使用 Canvas 绘制游戏画面

2. 支持键盘方向键控制蛇的移动

3. 蛇吃到食物后变长，分数增加

4. 撞到墙壁或自身则游戏结束

5. 界面美观，有分数显示和重新开始按钮

6. 所有代码写在一个 HTML 文件中

3.3 提示词自评

评价维度	评分	说明
清晰度	★ ★ ★ ★ ★	需求描述具体，包含 6 条明确要求
完整性	★ ★ ★ ★	涵盖了功能、界面、代码结构要求，但未指定响应式设计
可执行性	★ ★ ★ ★ ★	AI 可直接理解并执行，无需额外澄清
改进空间	—	可补充"添加音效"或"移动端触摸支持"等进阶要求

四、智能体提示词与论文解读

4.1 智能体提示词

你是一位资深银行业/金融科技领域学术论文解读助手。请对以下论文进行结构化解读：

【论文信息】

标题：{论文标题}

作者：{作者}

发表期刊：{期刊名称}

发表年份：{年份}

【解读要求】

1. 研究背景与动机：为什么做这个研究？解决了什么问题？
2. 核心研究方法：使用了什么模型/算法/数据？
3. 主要研究发现：得出了什么结论？
4. 创新点与贡献：相比已有研究有什么突破？
5. 局限性与未来方向：研究有哪些不足？
6. 实践启示：对银行业/金融科技有什么指导意义？

请用简洁清晰的语言，避免过度学术化表述。

4.2 论文基本信息

论文标题	Heterogeneous graph neural networks for fraud detection and explanation in supply chain finance
中文标题	用于供应链金融欺诈检测与解释的异构图神经网络
作者	Bin Wu（吴斌）、Kuo-Ming Chao（赵国明）、Yisheng Li（李亦生）
所属机构	复旦大学计算机科学学院 / 华威大学计算机科学系
发表期刊	Information Systems（Elsevier，CCF-B 类）
卷期/年份	Vol. 131, 2024, 文章编号 102335
DOI	https://doi.org/10.1016/j.is.2023.102335
研究领域	金融科技 / 图神经网络 / 欺诈检测 / 供应链金融
关键词	Fraud detection, Supply chain finance, Graph neural network, Multitask learning, Graph explainability, Heterogeneous graphs

4.3 研究背景与动机

现实问题

供应链金融市场近年来快速发展，中国供应链金融应收账款规模已达 23.11 万亿元，年增长率 9.7%。但该领域面临严峻的欺诈风险——核心企业、中小企业（SME）和金融机构之间的多层级交易关系复杂，欺诈者利用信息不对称伪造交易记录骗取融资。

现有研究的不足

单一视角局限：现有方法仅从单一角度分析，未同时考虑企业实体和交易行为

同构图假设缺陷：多数 GNN 方法将所有节点视为同一类型，忽略了企业节点与交易节点的异构性

缺乏可解释性：黑箱模型无法为风控人员提供决策依据

多任务分离：欺诈检测与解释通常作为独立任务处理

> **一句话概括痛点：**供应链金融中的欺诈检测需要同时"看懂"企业关系和交易行为，还要能说清楚为什么判定为欺诈。

4.4 研究问题与假设

编号	核心研究问题
RQ1	如何构建能够表达供应链金融中异构实体关系的图结构？
RQ2	如何在多视图（Multi-view）下同时学习企业和交易的表示？
RQ3	如何在实现高精度欺诈检测的同时，提供可解释的推理依据？

4.5 方法论（Methodology）

整体框架：MultiFraud

论文提出了 MultiFraud 多任务学习框架，包含四大模块：

▮ IV.B 异构图构建（Heterogeneous Graph Construction）

两类节点：企业节点 V（核心企业 + SME）+ 交易节点 P（转账、融资等）。
多种边类型。定义元路径（Metapath）如 Enterprise → Transaction → Enterprise 来捕获不同视角的关系。

▮ IV.C 多视图表示学习（Multi-view Representation Learning）

采用 Heterogeneous GNN 对不同类型的元路径分别编码。使用两层 GCN with skip connections，对每种元路径生成独立的节点嵌入后拼接。

IV.D 多实体嵌入共享 (Multi-entity Embedding Sharing)

引入 Attention-based 双向 LSTM 对交易时序建模，通过注意力机制让相关企业的嵌入相互增强。

IV.E 多任务欺诈检测器 (Multi-task Fraud Detector)

联合训练两个任务：企业分类（判断是否为欺诈企业）+ 交易分类（判断是否为欺诈交易）。损失函数 $L = \lambda L_{\text{enterprise}} + (1-\lambda)L_{\text{transaction}}$ ， $\lambda=0.5$ 平衡两任务权重。

多视图欺诈解释 (Multi-view Explanation)

基于 GNNExplainer 的模型无关方法，在多个视图上分别生成解释，输出关键特征权重（节点级）和重要边权重（边级）。

4.6 核心发现 (Key Findings)

编号	核心结论
F1	MultiFraud 在 5 个数据集上均优于现有 SOTA 方法，AUC 提升 2%~8%
F2	异构图建模显著优于同构图方法（Table 1 对比显示 FDGars、GraphConsi 等均不支持异构+多任务+解释三合一）
F3	注意力机制能有效识别高风险交易模式，解释结果符合业务逻辑
F4	多任务学习带来协同效应：企业检测和交易检测互相提升性能
F5	在宝武集团（Baowu）真实数据上

	的案例验证了框架的工业可用性
--	----------------

> 关键对比（Table 1）：现有方法中，只有 MultiFraud 同时具备
✓Heterogeneous + ✓Multitask + ✓Explainability 三大能力。

4.7 创新点与贡献

贡献类型	具体内容
理论贡献	首次系统性地提出供应链金融欺诈检测的异构图建模框架，填补了"多实体+多关系+可解释"三位一体的方法空白
方法贡献	提出 MultiFraud 框架：(1) 元路径驱动的多视图表示学习；(2) 注意力机制的多实体嵌入共享；(3) 多任务联合训练；(4) 多视图 GNN 解释器
实践意义	基于宝武集团数字资产平台真实数据验证，具有直接落地价值。金融机构可直接用于供应链金融风控系统升级

4.8 局限性分析与未来方向

作者承认的局限

当前框架主要针对应收账款融资场景，其他供应链金融产品（如预付款融资、存货融资）需适配

交易图的规模较大时（1.76 万亿条边），计算开销需要进一步优化

可延伸方向

引入动态图建模，捕捉时间演化模式

结合联邦学习解决跨机构数据隐私问题

扩展到跨境供应链的多币种/多监管场景

4.9 综合评价

> 一句话总结：本文提出的 MultiFraud 是首个同时实现异构图建模、多任务学习和可解释性的供应链金融欺诈检测框架，兼具学术创新性与工业落地潜力。

评分：★★★★☆ (4/5)

维度	评分	说明
问题重要性	★★★★★	23 万亿市场规模，欺诈防控刚需
方法创新性	★★★★★	首创异构+多任务+解释三合一框架
实验充分性	★★★★	5 个数据集 + 工业真实案例
写作质量	★★★★	结构清晰，图表规范
落地可行性	★★★★	已有工业界合作验证

推荐阅读人群：

商业银行供应链金融风控从业者

金融科技领域的 AI 算法工程师

研究图神经网络应用的研究生

关注金融反欺诈的技术管理者

五、Excel 合并日志与业绩预测报告

5.1 Excel 合并日志

一、源文件清单

序号	文件名	格式	原始行数	原始列数	列名变更情况
1	客户基本信息.xlsx	.xlsx	60	5	无变更
2	交易记录_v2.xlsx	.xlsx	80	5	Customer_ID → 客户 ID; trans_date → 交易日期
3	风险评估报告.xlsx	.xlsx	55	5	" 客户 ID " → 客户 ID; " 风 险评分 " → 风 险评分
4	产品持有信息.csv	.csv	70	5	cust_id → 客 户 ID
5	营销响应数据.xlsx	.xlsx	65	5	CID → 客户 ID; contact_dt → 联系日期
6	客户画像补充.xlsx	.xlsx	50	5	Client_ID → 客户 ID

二、列名归一化映射规则

不同来源文件对同一语义字段使用了不同的命名方式。以下是本次合并执行的归一化映射表：

标准列名	原始变体（来源文件）	归一化操作
客户 ID	客户 ID (客户基本信息); "客户 ID " (风险评估); Customer_ID (交易记录); cust_id (产品持有); CID (营销响应); Client_ID (客户画像)	strip + 统一重命名
交易/联系日期类	trans_date; 注册日期; 最近登录时间; 开户日期; contact_dt	保留原语义名称
金额类	Trans_Amount; 账户余额(元); discount_amt; 年总收入	保留原列名（含义不同）

三、合并过程详情

| 步骤 1: 文件读取

逐一读取 6 个文件，CSV 文件自动检测编码(utf-8-sig/gbk)，全部读取成功(6/6)

| 步骤 2: 列名归一化

对每张表的列名执行 strip() 去空格 + COLUMN_MAPPING 字典映射，共修改 7 处列名

| 步骤 3: 纵向拼接 Concat

使用 `pd.concat(ignore_index=True)` 将 6 个 DataFrame 纵向堆叠，合并前行数 =380

步骤 4: 去重

以「客户 ID + 交易日期」为联合主键执行 `drop_duplicates(keep=first)`，删除 169 条重复记录

步骤 5: 输出

最终输出 211 行 × 25 列，保存为 xlsx 格式

四、合并结果统计

指标	数值
源文件总数	6 个
合并前总行数	380 行
删除重复行数	169 行
最终总行数	211 行
最终总列数	25 列
输出文件大小	~30 KB (xlsx)

五、缺失值分析

由于采用 `outer join` 式的纵向拼接（各文件行数不等），合并后的宽表中存在大量结构性缺失。

缺失率等级	字段列表	说明
-------	------	----

0% (完整)	客户 ID	所有行均有值，作为主键
~62%	Trans_Amount, 交易日期, Transaction_Type, Merchant_Name	仅来自交易记录文件的 80 行有值
~73%	持有产品, 开户日期, 账户余额(元), 开户网点	仅来自产品持有信息的 57 行有值
~80%	客户姓名, 注册日期, 客户等级, 联系电话	仅来自基本信息的 42 行有值
~85%	campaign, response_flag, 联系日期, discount_amt	仅来自营销数据的 32 行有值
100%	年总收入, 职业类型, 所在城市, age, 风险评分, 信用等级, 逾期次数, 最近登录时间	完全缺失（需检查是否读取异常或确实无交集）

5.2 业绩预测报告

一、数据集概况

1.1 数据生成规则

本数据集为模拟生成的银行信用卡日消费额数据，具体构造规则如下：

- 时间跨度：2022 年 1 月 1 日至 2024 年 12 月 31 日，每日一 行，共 1096 条记录。
- 基础趋势：含线性上升趋势 + 年度正弦周期 + 周末效应 + 高斯随机噪声。
- 节假日效应：涵盖中国 22 个主要节假日（元旦/春节/清明/劳动/端午/中秋/国庆），节日期间及前后±1 天消费额随机放大 1.5~2.2 倍，共影响 124 天。
- 数据污染：故意制造约 5%缺失值(~54 个)和 1%异常值(~10 个 IQR 极端点)。

字段名	数据类型	说明	示例
日期	datetime	观测日期	2022-01-01
消费额(万元)	float	当日信用卡消费总额(万元)	1023.45
是否节日	str	是否在节假日期间	是/否
节日名称	str	对应节日名称(非空则表示节日)	春节 / (空)
星期几	str	星期几	周一 ~ 周日

1.2 数据质量摘要

质量指标	数值	处理方式
总记录数	1,096 天	-
缺失值数量	54 个 (4.9%)	前后 7 日滑动均值填补
异常值数量(IQR)	86 个 (7.8%)	标注但不删除
IQR 边界	[255.66, 1568.97] 万元	$Q1 - 1.5 \times IQR \sim Q3 + 1.5 \times IQR$
节假日天数	124 天 (11.3%)	含前后 ± 1 天放大窗口
日均消费额	约 900 万元	-

二、数据预处理

| 缺失值填补

采用前后各 7 天的滑动窗口均值进行填补。该方法能较好地保留局部趋势，避免全局均值带来的偏差。对于连续缺失超过 7 天的极端情况，回退至全局均值。

| 异常值检测与标注

使用 IQR（四分位距）方法检测异常值。计算公式：

- 下界 = $Q1 - 1.5 \times IQR$
- 上界 = $Q3 + 1.5 \times IQR$

检测到的异常值被标记为「偏高异常」或「偏低异常」，但原始值予以保留，供后续人工审核决策。

| 数据标准化(LSTM)

LSTM 模型训练前使用 MinMaxScaler 对消费额进行 [0,1] 归一化；预测输出后使用 inverse_transform 还原为原始量纲。SARIMA 模型直接使用原始数值。

训练/测试划分

最后 30 天 (2024-12-02 ~ 2024-12-31) 作为测试集，用于评估两种模型的预测精度。

三、EDA 探索性分析

图 1 日时序走势

消费额呈现明显的长期上升趋势（年均增长约 5%）和年度周期性（年中较高、年初较低）。周末消费显著高于工作日。红色标记显示异常值主要集中在年末大促期间。橙色下划线标记了原始缺失值的分布位置。

图 2 月度均值柱状图

月均消费额在 800~1100 万元区间波动。最高月份通常出现在 11-12 月（双 11/双 12 购物节叠加年底消费）；最低月份在 2 月（春节后消费低谷）。图中红色柱标注最高月份，绿色柱标注最低月份。

图 3 节日 vs 非节日箱线图

节日消费额的中位数和均值显著高于非节日。节日组箱体整体上移，且存在更多高值离群点。统计结果显示节日平均消费比非节日高出约 40%~70%，验证了节假日放大效应的有效性。

> EDA 图表已保存至: reports/eda/eda_three_plots.png

四、SARIMA 模型预测

4.1 模型配置

参数项	设定值	含义
模型类型	SARIMA	季节性自回归积分滑动平均模型
非季节阶数 (p,d,q)	(1, 1, 1)	AR=1, 差分阶=1, MA=1
季节阶数 (P,D,Q,s)	(1, 1, 1, 7)	季节 AR=1, 季节差分=1, 季节 MA=1, 周期=7 天
AIC	15,323.74	赤池信息量准则（越小越好）
BIC	15,348.52	贝叶斯信息量准则
训练集	1,066 天	2022-01-01 ~ 2024-12-01
测试集	30 天	2024-12-02 ~ 2024-12-31

4.2 预测结果

SARIMA 在测试集上的表现如下：

评价指标	数值	解读
RMSE	131.82 万元	预测值与实际值的均方根误差，越小越好
MAPE	11.88 %	平均绝对百分比误差，<15%属于良好水平
置信区间	95%	红色阴影区域为 95%置信带

> SARIMA 的优势在于能够显式建模周周期性 (*seasonal_order* 的 *s*=7)，这与信用卡消费的“周末效应”高度契合。

五、LSTM 深度学习模型预测

5.1 模型架构

超参数	设定值	说明
网络类型	LSTM (单层)	长短期记忆循环神经网络
输入窗口 (window_size)	14 天	用过去 14 天预测下一天
隐藏层单元 (hidden_size)	64	LSTM 隐层维度
训练轮次 (epochs)	20	完整遍历训练集 20 轮

批次大小 (batch_size)	32	每批 32 个样本
学习率 (lr)	0.001	Adam 优化器学习率
损失函数	MSELoss	均方误差损失
优化器	Adam	自适应矩估计优化器
训练样本量	1,052 个序列	1096-14(窗口)-30(测试)
最终 Loss	0.024	训练结束时 MSE(归一化空间)

5.2 训练过程

轮次	Loss 值	备注
Epoch 1	Loss = 0.343	初始状态，模型尚未收敛
Epoch 5	Loss = 0.276	快速下降阶段
Epoch 10	Loss = 0.197	持续优化
Epoch 15	Loss = 0.113	接近收敛
Epoch 20	Loss = 0.024	最终收敛，Loss 降低 93%

5.3 预测结果

评价指标	数值	解读
RMSE	400.29 万元	预测误差较大
MAPE	42.66 %	误差超过 40%，精度不足

> LSTM 表现不佳的可能原因: (1) 训练样本量偏少 (仅 1052 个序列) ; (2) 未针对季节性特征做专门工程设计; (3) epoch=20 可能不足以充分收敛; (4) 单层 LSTM 表达能力有限。实际生产中建议增加层数和数据量。

六、模型对比分析与结论

6.1 性能对比总表

对比维度	SARIMA(1,1,1)(1,1,1,7)	LSTM(w=14,h=64,ep=20)	胜出
RMSE (↓越低越好)	131.82	400.29	SARIMA
MAPE % (↓越低越好)	11.88%	42.66%	SARIMA

可解释性	★★★★★ 显式参数	★★☆☆☆ 黑箱模型	SARIMA
------	------------	------------	--------

6.2 结论

最优模型推荐: SARIMA(1,1,1)(1,1,1,7)

在本数据集上 RMSE 和 MAPE 两项指标全面优于 LSTM，且提供 95% 置信区间，更适合业务决策场景。

SARIMA 适用原因

信用卡消费具有强季节性（周周期 $s=7$ ）和趋势性，SARIMA 的差分+季节分解机制天然适配这类模式。

LSTM 改进方向

如要提升 LSTM 效果，建议：(1) 增加训练数据至 3 年以上；(2) 加入节假日哑变量作为外生特征；(3) 增至 2-3 层堆叠 LSTM；(4) 增加 epoch 至 100+ 并加入早停机制；(5) 尝试 Attention 机制增强长程依赖捕获。

业务应用建议

短期预测（未来 30 天）可采用 SARIMA 结果作为基准，结合业务经验调整。对于异常值（如促销活动导致的极端高点），建议人工复核后纳入特殊事件库。

七、输出文件索引

文件路径	说明
data/credit_card_daily.xlsx	原始模拟数据（含缺失值和异常值）
data/credit_card_processed.xlsx	处理后数据（已填补+已标注异常）
reports/eda/eda_three_plots.png	EDA 三图（时序/月度柱状/节日箱线）
reports/eda/prediction_comparison.png	SARIMA vs LSTM 预测对比折线图

六、实验心得

通过本次实验，我深入体验了 AI 辅助开发工具（TRAE）在软件开发全流程中的应用价值。以下是我的主要收获与反思：

一、AI 工具的效率提升

TRAE 的 AI 辅助功能显著提高了开发效率。在生成贪吃蛇游戏、银行 CRM 系统、财务报表等任务中，AI 能够在数秒内生成可运行的代码框架，相比传统手动编码方式，开发时间缩短了约 60%-70%。特别是在数据可视化（matplotlib 图表生成）和文档自动化（Word/PDF 报告生成）方面，AI 的表现尤为出色。

二、AI 出错案例与修正过程

在实验过程中，我遇到了一个典型的 AI 出错案例：在使用 python-pptx 生成 PPT 图表时，AI 生成的代码将颜色值以十六进制字符串格式（如 "#1a365d"）直接传递给 RGBColor 构造函数，导致运行时抛出 ValueError 异常。这是因为 RGBColor 需要的是 0-255 范围的整数元组，而非十六进制字符串。

修正方法：我通过分析错误堆栈信息，定位到问题出在颜色格式不匹配。随后我创建了两套颜色定义：COLORS 字典使用十六进制格式供 matplotlib 使用，RGB_COLORS 字典使用 (R, G, B) 元组格式供 python-pptx 和 python-docx 使用。这个修正过程让我深刻理解了不同库之间的 API 差异，也认识到 AI 生成代码后仍需人工审查和测试。

三、对 AI 辅助开发的思考

AI 工具并非万能，它在以下场景表现优异：

- 代码框架生成和样板代码编写
- 数据分析和可视化
- 文档格式化和报告生成
- 常见算法和模式实现

但在以下场景仍需人工介入：

- 复杂业务逻辑的设计
- 跨库 API 兼容性处理

- 性能优化和边界条件处理
- 安全敏感代码的审查

四、总结

本次实验让我认识到，AI 辅助开发工具是程序员的"超级助手"而非"替代者"。掌握如何有效使用 AI 工具（包括编写清晰的提示词、审查 AI 输出、快速定位和修复错误）已成为现代开发者的重要技能。未来我将继续探索 AI 在更多开发场景中的应用，同时保持对 AI 输出的批判性思维，确保代码质量和安全性。